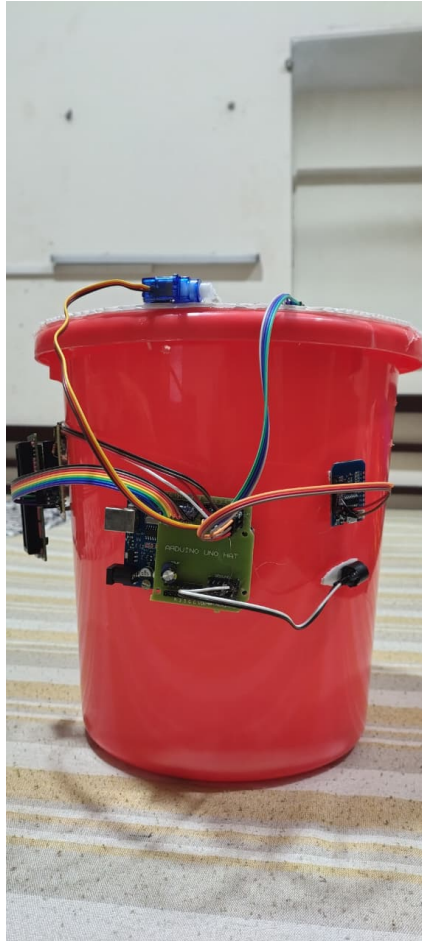


PROJECT REPORT

Smart Automated Dustbin

Touchless Lid Operation with Real-Time Fill-Level Monitoring



Working prototype of the Smart Automated Dustbin

Arduino Uno | HC-SR04 Ultrasonic Sensors | SG90 Servo | 16x2 LCD | Buzzer

Abstract

Waste bins are among the most frequently touched surfaces in homes, schools, hospitals and public spaces, and they are also among the least hygienic. Manual lid operation spreads germs, and bins that are never monitored overflow before anyone notices. This project presents a **Smart Automated Dustbin**, a low-cost embedded system that removes both problems.

The system is built around an **Arduino Uno**. A front-facing **HC-SR04 ultrasonic sensor** detects a hand or object approaching the bin and commands a **servo motor** to lift the lid, so waste can be dropped in without any physical contact. When the user moves away, the lid closes automatically. A second ultrasonic sensor mounted inside the lid measures the distance to the waste surface and converts it into a live **free-space percentage**, which is shown on a **16x2 LCD** together with the lid status. When free space drops to 10% or below, the system displays a **BIN FULL** message and sounds a **buzzer** so the bin can be emptied before it overflows.

The prototype was built on an ordinary plastic bin using inexpensive, widely available parts. Testing showed reliable touchless operation, consistent fill-level monitoring, and correct activation of the full-bin alert. The design is simple, affordable and easily extensible with wireless alerts, making it a practical building block for smart-city waste management.

Contents

1	Introduction
2	Background
3	Problem Definition
4	Objectives
5	System Components
6	System Design and Working Principle
7	Software Design
8	Prototype Assembly
9	Results and Discussion
10	Advantages and Applications
11	Limitations
12	Future Scope
13	Conclusion
A	Appendix: Complete Source Code

1. Introduction

Rapid urbanisation and growing populations have made waste management one of the most pressing challenges of modern cities. In most places, waste handling is still manual: lids are lifted by hand or foot, and bins are emptied on fixed schedules regardless of whether they are full. This exposes people to contaminated surfaces, causes unpleasant overflow in busy locations, and wastes collection effort on bins that are still nearly empty.

Embedded systems and sensing technology now make it possible to automate these routine tasks cheaply. A bin that opens by itself when approached eliminates contact with the lid, and a bin that knows how full it is can ask to be emptied at the right time.

This project implements exactly that: a **touchless, self-monitoring dustbin** built from low-cost, easily available components and controlled by an Arduino Uno. It combines two functions in one device: **proximity-based lid automation** and **continuous fill-level monitoring with local alerts**.

2. Background

Global waste generation is rising steadily, and the World Bank has projected that it could grow by nearly 70% by 2050 if left unmanaged. Traditional collection depends on manual labour and fixed routes, which is inefficient and unhygienic.

Smart waste management addresses this using sensors, microcontrollers and connectivity. Early smart bins only offered contactless lids using proximity sensors. Later designs added ultrasonic fill-level sensing, displays and wireless modules so that bin status could be monitored without inspection. The present project follows this path, combining touchless operation and fill-level monitoring on a single Arduino Uno platform at very low cost.

3. Problem Definition

Conventional dustbins suffer from the following problems:

- **Hygiene risk:** manual lid operation spreads germs, particularly in hospitals, schools and public places.
- **No visibility of fill level:** nobody knows a bin is full until it overflows.
- **Inefficient collection:** bins are emptied on schedule rather than on need.
- **Environmental impact:** overflowing bins attract pests and create odour and litter.

There is a need for an affordable system that can **detect a user automatically, open and close the lid without contact, measure how full the bin is, and alert people when it needs emptying**.

4. Objectives

- Enable **hygienic, contact-free** waste disposal.
- Use an ultrasonic sensor to **detect a user and operate the lid** automatically.
- **Measure the fill level** continuously and show it as a percentage on a display.

- **Alert users locally** (message and buzzer) when the bin is nearly full to prevent overflow.
- Keep the build **low-cost**, using common, easily sourced components.

5. System Components

Component	Role in the system
Arduino Uno (ATmega328P)	Main controller. Reads both sensors, drives the servo, LCD and buzzer, and runs the control logic.
HC-SR04 Ultrasonic Sensor (Sensor 1)	Mounted on the outside of the bin. Detects a hand or object approaching, triggering lid opening.
HC-SR04 Ultrasonic Sensor (Sensor 2)	Mounted inside the bin facing the waste. Measures distance to the waste surface for fill-level calculation.
Servo Motor (SG90 type)	Mounted at the lid. Rotates to 140 degrees to open and returns to 0 degrees to close the lid.
16x2 LCD (HD44780)	Displays the welcome message, lid status (OPEN/CLOSED), free-space percentage and the BIN FULL alert.
Buzzer	Gives a short beep on every lid movement and a longer alert tone when the bin is full.
Interface shield / wiring	Arduino Uno shield used to neatly route connections to sensors, servo, LCD and buzzer.
Power supply	Regulated supply (battery or adapter) for the Arduino and peripherals.
Plastic bin	Standard plastic container modified to hold the lid mechanism, sensors and electronics.

6. System Design and Working Principle

6.1 Block Diagram

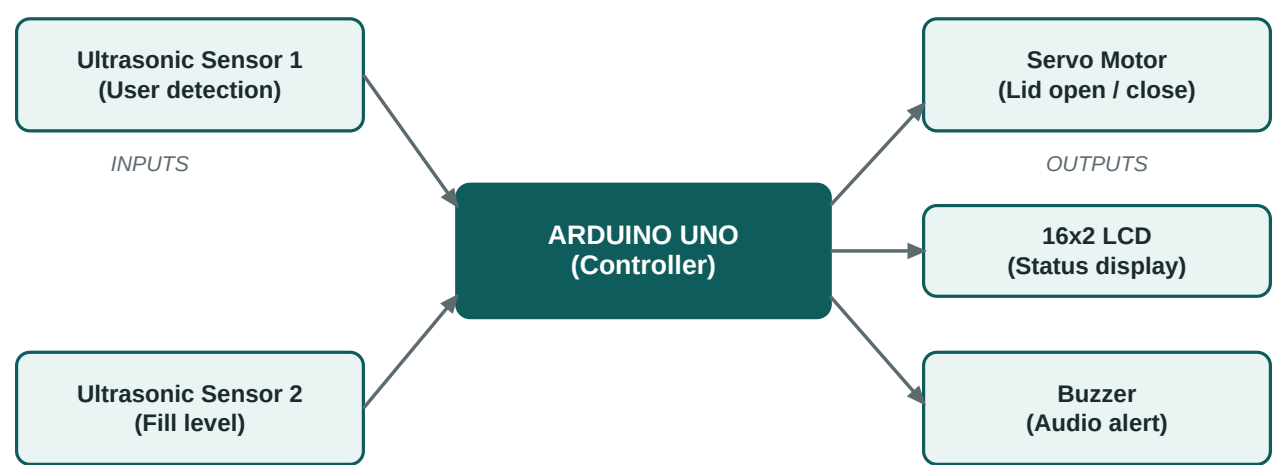


Figure 1: System block diagram

6.2 Working Principle

- **Start-up:** the system powers on, initialises the sensors, servo, LCD and buzzer, and moves the lid to the closed position. The LCD shows a welcome message for five seconds.
- **User detection:** Sensor 1 continuously measures the distance in front of the bin. When an object is detected closer than **30 cm**, the Arduino commands the servo to **140 degrees** and the lid opens, with a short confirmation beep.
- **Lid closing:** once the object moves away, the servo returns to **0 degrees** and the lid closes, again with a short beep.
- **Fill-level sensing:** Sensor 2, inside the bin, measures the distance to the top of the waste. A larger distance means more free space. The reading is converted to a free-space percentage and shown on the LCD.
- **Stabilisation:** the internal sensor is ignored for 1.5 seconds after every lid movement so the moving lid cannot cause false readings. The LCD shows "Calibrating..." during this time.
- **Full alert:** when free space is **10% or less**, the LCD displays "BIN FULL! EMPTY PLEASE" and the buzzer sounds, prompting the bin to be emptied.

6.3 Control Flow

Step	Action	Decision / Output
1	Read Sensor 1 (front)	Valid reading and distance < 30 cm?
2	If yes and lid is closed	Servo to 140 degrees, beep, record time, lid = OPEN
3	If no and lid was opened by a user	Servo to 0 degrees, beep, record time, lid = CLOSED
4	If 1.5 s have passed since last lid movement, read Sensor 2	Compute free % = (distance / 24 cm) x 100, capped at 100%
5	Update LCD and serial monitor	Show lid status and free %, or "Sensor Error" / "Calibrating..."
6	If free % is 10 or less	Show BIN FULL message and sound buzzer
7	Wait 500 ms	Repeat from step 1

6.4 Pin Configuration

Peripheral	Signal	Arduino Pin
Ultrasonic Sensor 1 (front)	TRIG / ECHO	D2 / D3
Ultrasonic Sensor 2 (internal)	TRIG / ECHO	D5 / D4
Servo motor	PWM signal	D6
LCD 16x2	RS, EN	D8, D9
LCD 16x2	D4, D5, D6, D7	D10, D11, D12, D13
Buzzer	Signal	A5

7. Software Design

The firmware is written in Arduino C++ using the standard *Servo* and *LiquidCrystal* libraries. Its key design features are described below.

7.1 Distance Measurement

The `readSensor()` function sends a 10 microsecond trigger pulse and measures the echo pulse width with a 30 ms timeout. Distance in centimetres is calculated as $\text{duration} \times 0.034 / 2$, based on the speed of sound. Readings outside 2 cm to 200 cm, or timeouts, are rejected and returned as an error value, so bad readings never move the lid or corrupt the fill level.

7.2 Lid Control Logic

The state variables `isOpen` and `wasNear` ensure the servo is commanded only when the state actually changes. This prevents the servo from being repeatedly re-triggered while a hand stays in front of the sensor, and avoids jitter.

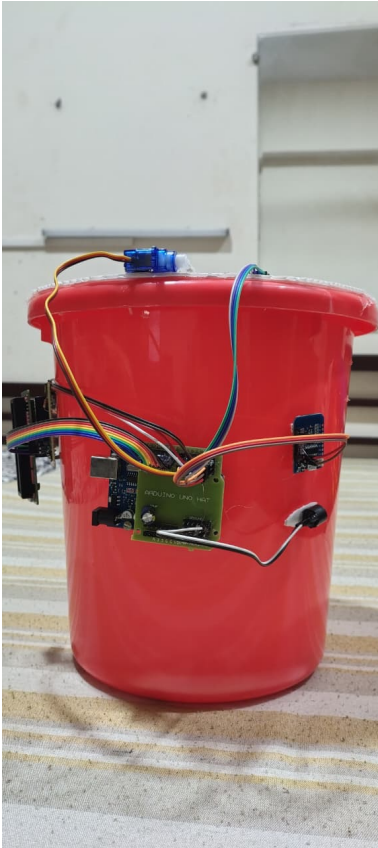
7.3 Fill-Level Calculation

The internal sensor measures the empty space between the lid and the waste. With the sensor-to-bottom height set to 24 cm, free space is $(\text{distance} \times 100) / 24$ per cent, limited to a maximum of 100%. The LCD shows a quick visual cue next to the value: a happy face above 50% free, neutral between 20% and 50%, and sad below 20%.

7.4 Reliability Features

- **Stabilisation delay (1.5 s)** after each lid movement avoids false fill-level readings from the moving lid.
- **Input validation** rejects out-of-range and timed-out sensor readings and shows "Sensor Error" on the display.
- **Serial debug output** prints both sensor distances, free percentage and lid events for easy calibration and testing.

8. Prototype Assembly



The prototype was assembled on a standard plastic bin. The main points of the build were:

- The **servo motor** is mounted at the top of the lid and connected to a PWM pin of the Arduino to swing the lid open and closed.
- The **front ultrasonic sensor** faces outward so it can detect a hand approaching the bin.
- The **internal ultrasonic sensor** is fixed inside the lid facing down onto the waste and calibrated for the bin height (24 cm).
- The **Arduino Uno**, with its interface shield, is mounted on the side of the bin and wired to the LCD, buzzer, sensors and servo.
- A **buzzer** provides audible feedback, and the **LCD** shows live status.
- All components run from a **regulated power supply** to avoid voltage fluctuations.

Figure 2: Assembled prototype showing the servo, controller, sensors and buzzer on the bin

9. Results and Discussion

After assembly and testing, the Smart Automated Dustbin performed as designed:

- **Touchless operation:** the lid opened reliably when a hand or object came within range and closed again after it moved away.
- **Fast response:** the delay between detection and lid opening was under one second.
- **Fill-level monitoring:** the LCD displayed a live free-space percentage, and the BIN FULL message and buzzer activated correctly when free space reached 10% or below.
- **Lighting independence:** because ultrasonic sensing was used instead of optical sensing, performance was consistent in varying light conditions.

Challenges and Solutions

Challenge	Solution
Sensor misalignment caused unreliable detection	Sensor positions and angles were adjusted for a clear detection zone.
False triggering of the lid	Code logic was refined with state tracking (isOpen / wasNear) and validated readings.
Internal sensor affected by lid movement	A 1.5 s stabilisation delay was added before fill-level readings are accepted.
Invalid or noisy ultrasonic echoes	Timeout and range checks reject bad readings and show a clear error message.

10. Advantages and Applications

Advantages

- Hygienic, completely touch-free disposal.
- Low cost, built from common and easily replaceable parts.
- Live fill-level information at a glance, with local alert before overflow.
- Simple, modular design that is easy to understand, repair and extend.
- Works regardless of lighting conditions.

Applications

- Hospitals and clinics, where infection control is critical.
- Schools, colleges and offices.
- Homes and kitchens.
- Public places such as parks, malls, railway stations and bus stops.
- Smart-city and campus waste-management initiatives.

11. Limitations

- The 24 cm bin height is a fixed calibration value and must be adjusted for bins of different sizes.
- Alerts are local only (LCD and buzzer); remote notification is not part of the current prototype.
- Ultrasonic sensing of very soft or angled waste surfaces can give slightly uneven readings.
- The system is intended for indoor or sheltered use and is not yet weatherproofed.

12. Future Scope

- **Wireless notifications** using a Wi-Fi module (such as ESP8266 / NodeMCU) to send a Telegram or mobile alert when the bin is full.
- **Mobile app and dashboard** for live monitoring of multiple bins.
- **Solar-powered operation** for outdoor and off-grid use.

- **Cloud data collection** to enable city-wide smart waste management and optimised collection routes.
- **AI-based prediction** of waste trends to schedule collection more efficiently.
- **Waste segregation** using additional sensors for dry, wet and recyclable waste.

13. Conclusion

The Smart Automated Dustbin shows how a simple embedded system can make an everyday object cleaner, smarter and more useful. Using an Arduino Uno, two ultrasonic sensors, a servo motor, an LCD and a buzzer, the project achieves contactless lid operation and live fill-level monitoring with a clear alert when the bin is nearly full. The prototype is inexpensive, reliable and easy to build, and it provides a solid base for wireless connectivity and large-scale smart waste management in future work.

Appendix: Complete Source Code

Arduino sketch (Arduino Uno). Libraries required: *LiquidCrystal* and *Servo* (both included with the Arduino IDE).

```
#include <LiquidCrystal.h>
#include <Servo.h>

const int rs = 8, en = 9, d4 = 10, d5 = 11, d6 = 12, d7 = 13;
LiquidCrystal lcd(rs, en, d4, d5, d6, d7);

#define TRIGGER_PIN_1 2
#define ECHO_PIN_1 3
#define TRIGGER_PIN_2 5
#define ECHO_PIN_2 4

Servo myservo;
int buz = A5;

bool isOpen = false;
bool wasNear = false;
long maxBinHeight = 24; // Total height from sensor to bottom
unsigned long lastLidActionTime = 0;
const unsigned long LID_STABILIZE_DELAY = 1500; // Wait after lid movement

void setup() {
  Serial.begin(9600);
  myservo.attach(6);
  pinMode(buz, OUTPUT);
  pinMode(TRIGGER_PIN_1, OUTPUT);
  pinMode(ECHO_PIN_1, INPUT);
  pinMode(TRIGGER_PIN_2, OUTPUT);
  pinMode(ECHO_PIN_2, INPUT);

  myservo.write(0);
  isOpen = false;

  lcd.begin(16, 2);
  lcd.setCursor(0, 0);
  lcd.print("  WELCOME");
  lcd.setCursor(0, 1);
  lcd.print("SMART DUST BIN");
  delay(5000); // show welcome screen for 5 seconds
  lcd.clear();
}

long readSensor(int triggerPin, int echoPin) {
  digitalWrite(triggerPin, LOW);
  delayMicroseconds(2);
  digitalWrite(triggerPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(triggerPin, LOW);

  long duration = pulseIn(echoPin, HIGH, 30000); // 30ms timeout
  if (duration == 0) {
    return -1; // Error reading
  }
}
```

```

    long distance = duration * 0.034 / 2;
    if (distance < 2 || distance > 200) {
        return -1;
    }

    return distance;
}

void loop() {
    long distance_1 = readSensor(TRIGGER_PIN_1, ECHO_PIN_1);

    long distance_2 = -1;
    int freePercentage = 0;

    if (millis() - lastLidActionTime > LID_STABILIZE_DELAY) {
        distance_2 = readSensor(TRIGGER_PIN_2, ECHO_PIN_2);

        if (distance_2 != -1) {
            freePercentage = (distance_2 * 100) / maxBinHeight;
            if (freePercentage > 100) freePercentage = 100;
        }
    }

    // Display
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Lid:");
    if (isOpen) {
        lcd.print("OPEN ");
    } else {
        lcd.print("CLOSED");
    }

    lcd.setCursor(0, 1);
    if (millis() - lastLidActionTime <= LID_STABILIZE_DELAY) {
        lcd.print("Calibrating... ");
    } else if (distance_2 == -1) {
        lcd.print("Sensor Error ");
    } else {
        lcd.print("Free:");
        lcd.print(freePercentage);
        lcd.print("% ");

        // Adjust emojis for free percentage (happy when more free space)
        if (freePercentage > 50) {
            lcd.print(":)"); // Happy when more than 50% free
        } else if (freePercentage > 20) {
            lcd.print(":|"); // Neutral when 20-50% free
        } else {
            lcd.print(":("); // Sad when less than 20% free
        }
    }
}

// Debug to Serial Monitor
Serial.print("Sensor1: ");
Serial.print(distance_1);
Serial.print("cm, Sensor2: ");
Serial.print(distance_2);
Serial.print("cm, Free: ");
Serial.print(freePercentage);
Serial.println("");

// Lid control - only if we got a valid reading
if (distance_1 != -1 && distance_1 < 30) {
    if (!isOpen) {
        myservo.write(140); // Open lid
        isOpen = true;
        lastLidActionTime = millis();
        digitalWrite(buz, HIGH);
    }
}

```

```

        delay(100);
        digitalWrite(buz, LOW);
        Serial.println("Lid OPENED");
    }
    wasNear = true;
} else {
    if (isOpen && wasNear) {
        myservo.write(0); // Close lid
        isOpen = false;
        lastLidActionTime = millis();
        digitalWrite(buz, HIGH);
        delay(100);
        digitalWrite(buz, LOW);
        Serial.println("Lid CLOSED");
    }
    wasNear = false;
}

// Bin full alert: free space of 10% or less
if (millis() - lastLidActionTime > LID_STABILIZE_DELAY &&
    distance_2 != -1 &&
    freePercentage <= 10) {

    digitalWrite(buz, HIGH);
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("BIN FULL!");
    lcd.setCursor(0, 1);
    lcd.print("EMPTY PLEASE");
    Serial.println("*** BIN FULL ALERT ***");
    delay(1000);
    digitalWrite(buz, LOW);
    delay(500); // Additional delay to prevent rapid retriggering
}

delay(500);
}

```